

FacturaExpress V3

Sistema de facturación electrónica con Angular y Node.js

GA8-220501096 - AA1 - EV02: Módulos integrados para la construcción del software

Aprendiz:	Jhon Henry Romero
Programa:	Análisis y Desarrollo de Software
Evidencia:	EV02 - Aplicación web integrada (frontend + backend) y documentación
Fecha:	Agosto de 2026

Repositorio del proyecto
https://github.com/jhromero81/FacturaExpress_V3.git

[CAPTURA]

Pantallazo sugerido: página principal del repositorio en GitHub mostrando el nombre, la descripción y las carpetas del proyecto.

1. Enlace del repositorio GitHub

1.1 Repositorio remoto

URL de clonación:	https://github.com/jhromero81/FacturaExpress_V3.git
Rama principal:	main
Control de versiones:	Git con historial de commits por funcionalidad El proyecto se gestiona con Git desde la fase inicial de codificación. Cada módulo se integra mediante commits descriptivos y el repositorio remoto en GitHub sirve como respaldo central y mecanismo de entrega continua.

1.2 Estructura del repositorio

```
FacturaExpress_V3/
|-- backend/                API REST Node.js + Express + MySQL
|   |-- routes/ controllers/ services/ middleware/ validators/
|   |-- config/ scripts/ test/ utils/      (setup DB, migraciones, seed)
|   |-- package.json          (scripts start/dev/test/db:*)
|-- frontend/              Aplicacion web Angular 22 (standalone)
|   |-- src/app/core/        (API, auth, guards, interceptor, render)
|   |-- src/app/features/    (12 modulos funcionales)
|   |-- proxy.conf.json      (/api -> http://localhost:4000)
|   |-- dist/                paquete de compilacion
|-- postman/collections/    Coleccion Postman v2.1 (56 peticiones)
|-- docs/                   Documentacion y evidencias
|-- README.md               Documentacion tecnica general
```

[CAPTURA]

Pantallazo sugerido: vista del explorador de archivos o de GitHub mostrando el arbol de carpetas del repositorio.

1.3 Instrucciones de clonado y ejecucion rapida

```
git clone https://github.com/jhromero81/FacturaExpress_V3.git

# Backend (puerto 4000)
cd FacturaExpress_V3/backend
npm install
copy .env.example .env      (y completar credenciales MySQL)
npm run db:setup             (crea la base de datos)
npm run db:migrate           (crea tablas)
npm run db:seed              (datos iniciales de prueba)
npm run dev                  (API en http://localhost:4000)

# Frontend (puerto 4200, proxia /api hacia 4000)
cd ../frontend
npm install
npm start                    (http://localhost:4200)
```

[CAPTURA]

Pantallazo sugerido: terminal con la salida de git clone y npm install exitosos en ambas carpetas.

2. Documentacion tecnica por modulo implementado

2.1 Arquitectura general de la solucion

```

+-----+ +-----+ +-----+
| FRONTEND Angular 22 | /api | BACKEND Express 4 | SQL | MySQL 8 |
| SPA standalone | ----> | API REST + JWT | ----> | 10+ tablas |
| http://localhost:4200 | proxy | http://localhost:4000 | | |
+-----+ +-----+ +-----+

```

La aplicacion web completa integra un frontend Angular 22 (componentes standalone, signals, router con lazy loading) con el backend REST Express ya desarrollado. En desarrollo, el dev-server de Angular redirige las peticiones /api al puerto 4000 mediante proxy.conf.json, lo que mantiene todo bajo el mismo origen y evita configuraciones CORS adicionales.

Autenticacion: POST /api/auth/login devuelve un JWT que el backend fija como cookie httpOnly; ademas el interceptor HTTP del frontend adjunta la cabecera Authorization: Bearer cuando existe token guardado.

Autorizacion por roles: guards de ruta (authGuard y adminGuard) en el frontend y middleware

authorize("admin") en el backend para los modulos administrativos.

Renderizado reactivo: el servicio ServicioRender (core/render.service.ts) registra la vista del modulo activo y, tras cada respuesta HTTP, ejecuta markForCheck() mas un ciclo de deteccion de cambios, garantizando que los datos aparezcan sin requerir interaccion del usuario.

Persistencia: transacciones atomicas para ventas (factura + items + descuento de stock), CUNE SHA-256 por factura y bitacora de auditoria de cada operacion.

2.2 Mapa de modulos de la aplicacion integrada

Modulo (ruta Angular)	Funcionalidad principal	Endpoints consumidos
/login	Inicio de sesion con validacion y manejo de errores	POST /auth/login
/dashboard	KPIs, grafico semanal y ultimas transacciones	/reportes/kpis, /ventas-semanales, /ultimas-transacciones
/ventas	Punto de venta: carrito, descuento y finalizacion	GET clientes-productos, POST /facturas
/facturas	Listado con filtros, paginacion y descargas	GET /facturas, GET /facturas/:id/{pdf,xml,csv}

/facturas/:id	Detalle, flujo DIAN (estado/firma/CUNE)	GET /facturas/:id, PUT /facturas/:id/estado
/clientes	CRUD de clientes con busqueda	GET/POST/PUT/DELETE /clientes
/productos	CRUD de productos y ajuste de inventario	GET/POST/PUT/DELETE/PATCH /productos
/reportes	Comparativo periodos, top productos e historial	/reportes/ventas-periodo, productos-top, historial, pdf
/configuracion	Empresa emisora, resolucion DIAN y sincronizacion	GET/PUT /configuracion, POST /configuracion/dian/sync
/usuarios (admin)	Gestion de cuentas y roles del sistema	GET/POST/PUT/PATCH/DELETE /usuarios
/errores (admin)	Monitoreo y marcado de errores internos	GET /errores, PATCH /errores/:id/resolver
/auditoria (admin)	Bitacora de operaciones por usuario/tabla/IP	GET /logs
/backup (admin)	Creacion, descarga, restauracion y borrado	GET/POST/DELETE /backup, POST /backup/restaurar

[CAPTURA]

Pantallazo sugerido: pantalla de inicio de sesion (login) con el diseno de la aplicacion.

2.3 Modulo de autenticacion y seguridad

Login contra /api/auth/login; contraseñas almacenadas con hash bcrypt.

- Sesión restaurada al recargar consultando /api/auth/me; cierre de sesión invalida el token en el servidor.
- Interceptor HTTP global (tokenInterceptor) que agrega Bearer token y redirige al login ante cualquier 401.
- Backend con helmet, rate-limit, express-validator y manejo centralizado de errores.

[CAPTURA]

Pantallazo sugerido: dashboard cargado tras iniciar sesión, con las tarjetas de KPIs visibles.

2.4 Modulo de ventas (punto de venta) y facturación

Selección de cliente, agregado de productos validando existencias y descuento porcentual (0-100%).

- Totales calculados igual que el servidor: subtotal, descuento, IVA 19% sobre base gravable y total.
- Al finalizar, el backend genera número consecutivo y CUNE, descuenta stock en transacción atómica y deja la factura pendiente ante la DIAN.
- Descarga comprobable en tres formatos: PDF (documento soporte), XML DIAN y CSV de reporte.

[CAPTURA]

Pantallazo sugerido: pantalla de Nueva venta con productos en el carrito y totales calculados.

[CAPTURA]

Pantallazo sugerido: detalle de una factura emitida mostrando CUNE y botones PDF/XML/CSV.

2.5 Modulo de clientes y productos

Clientes: creación, edición, búsqueda incremental y eliminación lógica; validaciones de campos obligatorios y correo.

- Productos: catálogo con precio, tarifa de IVA (0/5/19%) e inventario; ajuste de stock (+/-) con PATCH / productos/:id/stock.

Ambos listados muestran insignias de estado (stock crítico/bajo/ok) y mensajes de confirmación.

[CAPTURA]

Pantallazo sugerido: listado de clientes y modal de creación de cliente.

[CAPTURA]

Pantallazo sugerido: listado de productos con el modal de ajuste de stock abierto.

2.6 Modulo de reportes y configuración fiscal

Reportes: comparativo de ventas del periodo actual vs anterior (semanal/mensual/trimestral/anual), top 5

- de productos más vendidos y generación de reporte PDF con historial persistente.

Configuracion: datos de la empresa emisora (NIT, razon social, email, telefono), resolucion DIAN, vigencia del certificado y boton de sincronizacion con la DIAN que actualiza ultima_sync.

[CAPTURA]

Pantallazo sugerido: vista de reportes con grafico comparativo y tabla de productos top.

[CAPTURA]

Pantallazo sugerido: pantalla de configuracion tras ejecutar la sincronizacion DIAN.

2.7 Modulos administrativos (solo rol admin)

Usuarios: alta/edicion con roles (admin, vendedor, contador), activacion/desactivacion y eliminacion protegida.

Errores del sistema: KPIs resueltos/sin resolver, filtros por tipo (firma, dian, bd, correo) y marcado de resolucion.

Auditoria: bitacora con usuario, accion, tabla afectada, registro e IP de origen.

Backup: creacion de respaldos SQL, descarga, restauracion con doble confirmacion y eliminacion.

[CAPTURA]

Pantallazo sugerido: gestion de usuarios con el modal de nuevo usuario.

[CAPTURA]

Pantallazo sugerido: vistas de errores, auditoria y backup del sistema.

2.8 Paquete de compilacion del frontend (Angular)

```
cd frontend
npm run build          (produccion)

Salida:
dist/facturaexpress-frontend/browser/
|-- index.html
|-- main-*.js          bundle inicial (~112 kB / ~29 kB gzip;
                        total inicial ~354 kB / ~99 kB gzip)
|-- chunk-*.js         chunks diferidos por modulo (lazy loading)
`-- styles-*.css       sistema de diseno global
```

El build de produccion compila sin errores ni advertencias. Los componentes se cargan de forma diferida (loadComponent), generando chunks independientes por modulo, lo que optimiza la carga inicial de la aplicacion.

[CAPTURA]

Pantallazo sugerido: terminal con el resultado de ng build mostrando los chunks y el mensaje Application bundle generation complete.

3. Informe de las pruebas de software realizadas

3.1 Pruebas unitarias (node --test)

El backend incluye una suite de pruebas unitarias nativas de Node.js sobre las utilidades criticas: generacion del CUNE (SHA-256 determinista), helpers de formato y calculo de IVA, firma/verificacion de JWT, middleware de validacion y reglas de negocio de los validadores de entrada.

```
cd backend
npm test
```

```
Resultado: 40 pruebas unitarias OK en 5 archivos
  cune.test.js  helpers.test.js  jwt.test.js  validate.test.js  validators.test.js
```

[CAPTURA]

Pantallazo sugerido: terminal con la ejecucion de npm test mostrando los 40 tests aprobados.

3.2 Pruebas de integracion de la API (Postman / Newman)

La coleccion postman/collections/FacturaExpress-API-Express.postman_collection.json cubre los 12 modulos del backend con 56 peticiones y 91 aserciones automatizadas (codigos HTTP, esquema JSON, persistencia en base de datos, seguridad y casos negativos).

```
npx newman run postman/collections/FacturaExpress-API-Express.postman_collection.json \
-e postman/environment.json --reporters cli,json
```

Peticiones ejecutadas

Aserciones evaluadas

Fallos

56

91

0

[CAPTURA]

Pantallazo sugerido: resumen final de Newman mostrando iterations/requests/assertions y 0 failures.

3.3 Verificacion end-to-end frontend <-> backend

Prueba integral ejecutada con el dev-server de Angular (puerto 4200) y su proxy hacia la API (puerto 4000), replicando exactamente las peticiones que emite el navegador. Adicionalmente se verifico el build de produccion servido estaticamente en http://localhost:3000 con un proxy /api hacia el backend:

Escenario verificado

Resultado

Login por proxy con cookie httpOnly + rol admin

OK

KPIs del dashboard (facturas emitidas, ventas)	OK
Listados de clientes (8) y productos (10)	OK
Venta POS: subtotal 325.000, descuento 10% (32.500), IVA 55.575, total 348.075	OK
Descuento atomico de stock (-1 y -2 unidades)	OK
Detalle de factura con CUNE generado	OK
Descarga PDF (2.351 B), XML (956 B) y CSV (304 B)	OK
Cambio de estado DIAN: enviada, firma firmada, 1 intento	OK
Endpoints admin: usuarios (9), errores, logs (61), backups (2)	OK
Endpoints reportes: periodo, top, historial y PDF (2.211 B)	OK
Renderizado automatico de datos en los 11 modulos navegables sin interaccion del usuario (verificacion con navegador headless)	OK
Graficos SVG funcionales: ventas semanales (7 barras) y comparativo por periodos con cambio Semanal/Mensual/Anual	OK
Los calculos de la venta de prueba coinciden con la logica del servidor (descuento porcentual sobre el subtotal e IVA del 19% sobre la base gravable), confirmando la consistencia entre frontend y backend. La verificacion de renderizado confirma el correcto funcionamiento de ServicioRender y el backend	

[CAPTURA]

Pantallazo sugerido: navegador mostrando la factura creada y su PDF descargado.

4. Manual tecnico de instalacion y ejecucion

4.1 Requisitos previos

- Node.js 20 o superior (verificado con v26.7.0) y npm 10+.
- MySQL Server 8.x en ejecucion local con usuario con permisos de creacion de bases de datos.
- Opcional para pruebas de API: Postman o Newman (npx newman).

4.2 Configuracion del backend

Crear el archivo backend/.env a partir de .env.example con los valores correspondientes:

```
PORT=4000
CORS_ORIGIN=http://localhost:4200
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=<su-clave-mysql>
DB_NAME=facturaexpress_apirest
JWT_SECRET=<frase-secreta-larga>
JWT_EXPIRES_IN=8h
```

4.3 Base de datos, migraciones y datos iniciales

```
cd backend
npm run db:setup      crea la base facturaexpress_apirest
npm run db:migrate    crea tablas (empresa, usuarios, clientes, productos,
                      facturas, factura_items, logs_auditoria, errores_sistema,
                      backups, reportes)
npm run db:seed       carga empresa demo, usuarios, clientes y catalogo
```

Usuario administrador de demostracion creado por el seed: NIT 900.123.456-7, contrasena admin123.

4.4 Ejecucion de la aplicacion integrada

```
# Terminal 1 - API REST (http://localhost:4000/api/health para validar)
cd backend && npm run dev          (nodemon, recarga automatica)

# Terminal 2 - Frontend Angular (http://localhost:4200)
cd frontend && npm start           (dev-server con proxy /api -> 4000)
```

Abrir <http://localhost:4200> e iniciar sesion con el usuario demo. El menu lateral muestra los modulos principales para todos los roles y la seccion de administracion solo para el perfil admin.

4.5 Compilacion de produccion

```
cd frontend
npm run build
-> dist/facturaexpress-frontend/browser    (paquete estatico listo para publicar)
```

4.6 Solucion de problemas frecuentes

Error ECONNREFUSED al iniciar el backend: verificar que MySQL este activo y las credenciales del .env

- sean correctas.
- Puerto 4000 o 4200 ocupado: liberar el proceso (Get-Process node / netstat -ano) o cambiar PORT en .env / --port en ng serve.
- npm no instala dependencias del frontend (politica allow-scripts de npm 11): el package.json del frontend ya incluye la seccion allowScripts con esbuild/rollup; usar npm install dentro de frontend/.
- Respuestas 401 tras mucho tiempo de inactividad: la sesion expiro (JWT 8h); volver a iniciar sesion.
- Datos antiguos o pantalla incompleta tras actualizar el frontend: recargar con Ctrl+F5; el index.html se entrega sin cache y los assets hashados se sirven como inmutables.
- No se descarga el PDF/XML/CSV: confirmar que el backend este corriendo y que la factura exista; revisar la pestana de red del navegador.

Conclusiones

La aplicacion web queda completamente funcional e integrada: 12 modulos de interfaz Angular

- consumiendo los servicios REST reales del backend con persistencia en MySQL.
- Las tres capas de verificacion (unitarias, integracion de API y end-to-end) reportan resultados satisfactorios: 40/40 pruebas unitarias, 91 aserciones sin fallos y los flujos de negocio verificados en el navegador.
- El renderizado de datos es automatico en todos los modulos (ServicioRender + backend XHR), verificado con navegador headless sin interaccion del usuario.
- El repositorio GitHub documenta la evolucion del software y contiene el paquete de compilacion del frontend requerido por la evidencia (frontend/dist).

